

24-3-2026

MANUAL TÉCNICO

SISTEMA DE RECURSOS CEAA

COMISIÓN ESTATAL DEL AGUA Y
ALCANTARILLADO

VERSIÓN 1.0 MARZO 2026

Camino Real de la Plata No. 336,
Zona Plateada, Pachuca de Soto, Hgo. C. P. 42084
Ofic.: 771 715 8390 y 771 715 8391
ceaa.hidalgo.gob.mx

Contenido

1. DESCRIPCIÓN GENERAL DEL SISTEMA.....	4
2. REQUISITOS DEL SERVIDOR	4
3. ESTRUCTURA DE DIRECTORIOS DEL PROYECTO	5
4. CONFIGURACIÓN DEL ENTORNO	9
4.1 config/config.php	9
4.2 config/database.php	10
5. ARQUITECTURA DE LA APLICACIÓN (MVC)	12
5.1 Front Controller y Router (public/index.php).....	12
5.2 Autoload de clases	14
5.3 Flujo de una petición HTTP	14
6. MÓDULO DE AUTENTICACIÓN Y SESIONES.....	15
6.1 Inicio de sesión (AuthController::doLogin).....	15
6.2 Cierre de sesión (AuthController::logout)	17
6.3 Protección de controladores	17
7. SISTEMA DE ROLES Y PERMISOS	18
8. PROTECCIÓN CSRF	19
9. CAPA DE ACCESO A DATOS (MODELOS).....	20
9.1 Clase DB (config/database.php).....	20
9.2 Modelo User (models/User.php).....	21
9.3 Modelo Inventario (models/Inventario.php)	22
9.4 Modelo EcaFicha (models/EcaFicha.php)	23
9.5 Modelo Catalogo (models/Catalogo.php)	24
9.6 Otros modelos	25
10. CONTROLADORES	26
10.1 AuthController	26
10.2 DashboardController	27



10.3 UserController	27
10.4 FormatoController	29
10.5 InventarioController.....	30
10.6 ReporteController.....	31
10.7 ManualController.....	32
10.8 ChatbotController	33
11. VISTAS Y LAYOUT	34
11.1 Plantilla maestra (views/dashboard.php).....	34
11.2 Organización de vistas	35
11.3 Carga condicional de hojas de estilo	36
12. GENERACIÓN DE PDFs	37
12.1 Reportes municipales.....	37
12.2 Reportes anuales (controllers/export/InventarioFormatoAnualPDF.php) ...	38
12.3 Ficha técnica ECA (controllers/export/pdf_eca.php)	39
13. EXPORTACIÓN A EXCEL (PhpSpreadsheet)	40
14. SISTEMA DE BITÁCORA (AUDITORÍA).....	41
14.1 Clase Bitacora (models/Bitacora.php).....	41
14.2 Tabla bitacora_acciones	42
14.3 Eventos registrados por módulo	43
15. SUBIDA DE ARCHIVOS	44
16. ESQUEMA DE BASE DE DATOS.....	45
16.1 Tablas principales.....	45
16.2 Relaciones clave.....	45
17. DEPENDENCIAS EXTERNAS (COMPOSER)	46
18. ESTRUCTURA DE ASSETS	48
19. SCRIPTS DE MANTENIMIENTO	49
20. SEGURIDAD APLICADA.....	50



21. ERRORES COMUNES Y SOLUCIONES.....	51
22. GUÍA DE DESPLIEGUE Y ACTUALIZACIÓN	53

Ilustración 1- config.php	9
Ilustración 2- constante definida	10
Ilustración 3- database.php	11
Ilustración 4- mapa de controles.....	14
Ilustración 5- AuthController.php(login)	16
Ilustración 6- User.php	21
Ilustración 7- Inventario.php	22
Ilustración 8- EcaFicha.php	23
Ilustración 9- Catalogo.php	24
Ilustración 10- Controlador.....	26
Ilustración 11- UserController.php	27
Ilustración 12- Formato Cotroller	29
Ilustración 13- Inventario Controller.....	30
Ilustración 14- Reporte controller	31
Ilustración 15- Manual controller	32
Ilustración 16- ChatBot Controller	33
Ilustración 17- Pantilla maestra(dashboard.php).....	34
Ilustración 18- InventarioFormatoMunicipal.php.....	37
Ilustración 19- InventarioFormatoAnual.php.....	38
Ilustración 20- pdf_eca.php	39
Ilustración 21- Bitacora.php.....	41



1. DESCRIPCIÓN GENERAL DEL SISTEMA

El Sistema de Recursos CEAA es una aplicación web desarrollada en PHP puro siguiendo el patrón de diseño MVC (Modelo-Vista-Controlador), sin uso de frameworks externos.

- Lenguaje: PHP 8.x
- Base de datos: MariaDB 10.4 / MySQL 8.x
- Servidor web: Apache 2.4 (XAMPP)
- Dependencias: phppoffice/phpspreadsheet, dompdf/dompdf, mpdf/mpdf
- Gestión deps: Composer
- Base URL: http://localhost/ceaa_recursos/public/
- Nombre BD: sistema_ceaa

2. REQUISITOS DEL SERVIDOR

Software requerido:

- PHP 8.0 o superior
- Extensiones PHP: pdo, pdo_mysql, mbstring, gd, fileinfo, zip, curl, openssl, json, intl
- Apache 2.4+ con mod_rewrite habilitado
- MariaDB 10.4+ o MySQL 8.0+
- Composer 2.x (para instalar y actualizar dependencias)

Hardware mínimo:



- 2 GB de RAM disponibles
- 1 GB de espacio en disco para la aplicación y PDFs generados

Recomendado en producción:

- Habilitar HTTPS (SSL/TLS)
- PHP con OPcache habilitado para mejor rendimiento
- Límite de memoria PHP: 256M o superior (requerido por mPDF)

3. ESTRUCTURA DE DIRECTORIOS DEL PROYECTO

ceaa_recursos/

— composer.json	Dependencias del proyecto
— vendor/	Librerías instaladas por Composer
— config/	
— config.php	Configuración general (app_name, base_url)
— database.php	Clase DB — conexión PDO a MySQL
— helpers.php	Funciones globales (asset(), csrf_token(), csrf_validate(), require_role())
— controllers/	
— AuthController.php	Login, logout
— DashboardController.php	Inicio, perfil, cambio de contraseña
— UserController.php	CRUD usuarios (solo admin)



		FormatoController.php	Fichas ECA, captura, consulta, edición, PDF
		InventarioController.php	CRUD inventario de bienes
		ReporteController.php	Lista y elimina reportes municipales/anuales
		ManualController.php	Sirve el manual técnico en PDF
		ChatbotController.php	Respuestas del asistente virtual
		actualizar_perfil.php	Acción AJAX para actualizar datos de perfil
		cambiar_password.php	Acción AJAX para cambiar contraseña
		export/	
		InventarioFormatoAnual.php	Excel inventario anual
		InventarioFormatoAnualPDF.php	PDF inventario anual
		InventarioFormatoMunicipio.php	Excel inventario municipal
		InventarioFormatoMunicipioPDF.php	PDF inventario municipal
		pdf_eca.php	PDF ficha técnica ECA
		models/	
		Bitacora.php	Registro de auditoría en BD
		Catalogo.php	Catálogos (categorías, unidades, organismos...)
		EcaFicha.php	CRUD fichas técnicas ECA
		Formato.php	Formatos genéricos
		FormatoModel.php	Lógica de formatos PDF
		Inventario.php	CRUD bienes del inventario
		Municipio.php	Catálogo de municipios



— Organismo.php	Catálogo de organismos
— Reporte.php	Gestión de reportes generados
— Role.php	Catálogo de roles del sistema
— User.php	CRUD usuarios
— views/	
— dashboard.php	Plantilla maestra (layout principal)
— auth/	
— login.php	
— dashboard/	
— inicio.php	
— perfil.php	
— formatos/	
— index.php	Dashboard de formatos con Chart.js
— captura_eca.php	Alta de nueva ficha ECA
— eca_consulta.php	Búsqueda y listado de fichas
— ver_eca.php	Detalle de ficha (solo lectura)
— editar_eca.php	Edición de ficha ECA
— reporte.php	Generador de reportes PDF
— inventario/	
— captura.php	Alta rápida de bien
— form.php	Formulario completo de bien



— layouts/	Vistas parciales reutilizables
— reportes/	
— municipales.php	Listado de reportes municipales
— anuales.php	Listado de reportes anuales
— inventario.php	Inventario con búsqueda y edición modal
— users/	
— index.php	Lista de usuarios
— create.php	Formulario de nuevo usuario
— form.php	Formulario de edición
— public/	
— index.php	Front Controller — punto de entrada único
— assets/	
— css/	Hojas de estilo por módulo
— js/	Scripts JavaScript
— img/	Imágenes estáticas
— formatos/	Plantillas de formatos
— uploads/perfiles/	Fotos de perfil de usuarios
— sql_updates/	Scripts SQL de actualizaciones de esquema
— scripts/	Scripts de migración y utilidades
— Chatbotweb/	Módulo del chatbot (independiente)
— index.php	



└─ enviar-correo.php

└─ vendor/ Librerías del chatbot (PHPMailer)

4. CONFIGURACIÓN DEL ENTORNO

4.1 config/config.php

Ilustración 1- config.php

Retorna un array asociativo con configuración básica:

```
<?php  
return [  
    'app_name'    => 'CEAA — Recursos',  
    'base_url'    => rtrim(dirname($_SERVER['SCRIPT_NAME']), '/'),  
    'session_name' => 'ceaa_sess'  
];
```

PARÁMETROS

app_name Nombre de la aplicación (mostrado en el título del navegador).

`base_url` URL base calculada dinámicamente desde el servidor. En
XAMPP local: `/ceaa_recursos/public`

`session_name` Nombre de la cookie de sesión.

CONSTANTE DEFINIDA

en `public/index.php`:

```
define('BASE_URI', rtrim(dirname($_SERVER['SCRIPT_NAME']), '/'))
```

Disponible globalmente. Se usa para construir URLs absolutas dentro del sistema.

```
public > index.php > ...
1 <?php
2 date_default_timezone_set('America/Mexico_City');
3 session_start();
4
5 /* Helpers de ruta para assets */
6 define('BASE_URI', rtrim(dirname($_SERVER['SCRIPT_NAME']), '/')); // p.ej: /ceaa_recursos/public
7 function asset(string $path): string {
8     return BASE_URI . '/assets/' . ltrim($path, '/');
9 }
10
11 /* Autoload: busca en /app/* y también en la raíz (/controllers, /models) */
12 spl_autoload_register(function ($class) {
13     $roots = [
14         __DIR__ . '/../app/controllers/',
15         __DIR__ . '/../app/models/',
16         __DIR__ . '/../controllers/',
17         __DIR__ . '/../models/',
18     ];
19     foreach ($roots as $dir) {
20         $file = $dir . $class . '.php';
21         if (is_file($file)) {
22             require_once $file;
23             return;
24         }
25     }
26 }
```

Ilustración 2- constante definida

4.2 config/database.php

Contiene la clase DB con patrón Singleton para la conexión PDO:

```
class DB {
```



```
private static $pdo = null;

public static function conn() {

    if (!self::$pdo) {

        $dsn = 'mysql:host=localhost;dbname=sistema_ceaa;charset=utf8mb4';

        self::$pdo = new PDO($dsn, 'root', '', [

            PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,

            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,

        ]);

    }

    return self::$pdo;

}

}
```

```
config > database.php > ...
1  <?php
   50 references | 0 implementations
2  class DB {
   3 references
3      private static $pdo = null;
   47 references | 0 overrides
4      public static function conn() {
5          if (!self::$pdo) {
6              $dsn = 'mysql:host=localhost;dbname=sistema_ceaa;charset=utf8mb4';
7              self::$pdo = new PDO($dsn, 'root', '', [
8                  PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
9                  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
10             ]);
11         }
12         return self::$pdo;
13     }
14 }
15
```

Ilustración 3- database.php



ATRIBUTOS DE CONEXIÓN

host localhost

dbname sistema_ceaa

charset utf8mb4 (soporta caracteres especiales y emojis)

usuario root

password (vacío — ajustar en producción con credenciales seguras)

errmode ERRMODE_EXCEPTION — las fallas de BD lanzan excepciones
PHP

fetch_mode FETCH_ASSOC — los resultados se obtienen como array
asociativo

IMPORTANTE PARA PRODUCCIÓN:

- Cambiar 'root' y " por credenciales del usuario de BD con permisos mínimos necesarios (solo SELECT/INSERT/UPDATE/DELETE en sistema_ceaa).
- Nunca usar root sin contraseña en un servidor de producción.
- Considerar mover credenciales a variables de entorno o archivo .env.

5. ARQUITECTURA DE LA APLICACIÓN (MVC)

5.1 Front Controller y Router (public/index.php)

Todas las peticiones web deben entrar a través de:

http://servidor/ceaa_recursos/public/index.php



El enrutamiento usa dos parámetros GET:

- controller : identifica qué controlador manejar la petición
- action : identifica qué método del controlador ejecutar

Ejemplo de URL:

index.php?controller=formatos&action=capturaECA

→ Instancia FormatoController y llama al método capturaECA()

MAPA DE CONTROLLERS:

controller → Clase PHP

auth → AuthController

dashboard → DashboardController

users → UserController

formatos → FormatoController

inventario → InventarioController

reportes → ReporteController

reporte → ReporteController (alias)

manual → ManualController

chatbot → ChatbotController

Si el controlador o la acción no existen, el sistema responde con HTTP 404.



```
require_once __DIR__ . '/../config/config.php';
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../config/helpers.php';

/* Router */
$controller = $_GET['controller'] ?? 'auth';
$action      = $_GET['action'] ?? 'login';

$map = [
    'auth'       => 'AuthController',
    'dashboard'  => 'DashboardController',
    'users'      => 'UserController',
    'formatos'   => 'FormatoController',
    'inventario' => 'InventarioController',
    'reportes'   => 'ReporteController',
    'reporte'    => 'ReporteController', //
    'manual'     => 'ManualController',
    'chatbot'    => 'ChatbotController',
    'bitacora'   => 'BitacoraController',
];
```

Ilustración 4- mapa de controles

5.2 Autoload de clases

Se registra un autoloader con `spl_autoload_register` que busca archivos de clase en estas rutas (en orden):

1. `__DIR__ . '/../app/controllers/'`
2. `__DIR__ . '/../app/models/'`
3. `__DIR__ . '/../controllers/'`
4. `__DIR__ . '/../models/'`

Para añadir una nueva clase basta con crear el archivo `NombreClase.php` en la carpeta `controllers/` o `models/`, con `class NombreClase { }` adentro. El sistema la cargará automáticamente sin configuración adicional.

5.3 Flujo de una petición HTTP

1. Navegador envía GET/POST a `public/index.php`



2. index.php inicia sesión (session_start)
3. index.php define BASE_URI y la función asset()
4. index.php carga config.php, database.php, helpers.php
5. index.php lee ?controller y ?action de \$_GET
6. index.php busca la clase en el mapa y la instancia
7. index.php llama al método \$action() en la instancia
8. El método del controlador:
 - a. Verifica autenticación / rol (guardar o redirigir)
 - b. Llama a modelos para obtener datos
 - c. Llama a la vista con extract() + require
9. La vista incluye dashboard.php (layout) que inyecta el HTML final
10. El HTML se devuelve al navegador

6. MÓDULO DE AUTENTICACIÓN Y SESIONES

6.1 Inicio de sesión (AuthController::doLogin)



```
AuthController.php M X
controllers > AuthController.php > ...
1 <?php
2 0 references | 0 implementations
3 class AuthController {
4     1 reference
5     private function saludoPorHorario(string $nombre): string {
6         $hora = (int)date('G');
7
8         if ($hora < 12) {
9             return "Buenos días, {$nombre}. Bienvenid@ al sistema.";
10        }
11
12        if ($hora < 19) {
13            return "Buenas tardes, {$nombre}. Bienvenid@ al sistema.";
14        }
15
16        return "Buenas noches, {$nombre}. Bienvenid@ al sistema.";
17    }
18
19    1 reference
20    private function render($view, $params=[]){
21        extract($params);
22        ob_start();
23        require __DIR__."/../views/$view.php";
24        return ob_get_clean();
25    }
26
27    0 references | 0 overrides
28    public function login()
```

Ilustración 5- AuthController.php(login)

Flujo:

1. POST a ?controller=auth&action=doLogin con email y password.
2. Se busca el usuario con User::findByEmail(\$email).
3. Se verifica con password_verify(\$pass, \$user['password_hash']).
4. Se verifica que el campo activo = 1.
5. Si falla cualquier paso → se registra en bitácora como 'login_fallido' y se redirige al login con mensaje flash.
6. Si todo es correcto → se almacena en sesión:

```
$_SESSION['user'] = [
    'id'      => $user['id'],
    'nombre'  => $user['nombre'],
    'email'   => $user['email'],
```



```
'rol_id'    => $user['rol_id'],  
  
'foto_perfil' => $user['foto_perfil'] ?? '',  
  
];
```

7. Se registra en bitácora como 'login' y se redirige al dashboard.

Las contraseñas se almacenan con PHP PASSWORD_DEFAULT (bcrypt). Nunca se almacena ni transmite la contraseña en texto plano.

6.2 Cierre de sesión (AuthController::logout)

1. Se registra en bitácora como 'logout'.
2. Se destruye la sesión con session_destroy().
3. Se redirige al login.

6.3 Protección de controladores

Cada controlador que requiere autenticación llama a uno de estos helpers definidos en config/helpers.php:

require_role([ROL_ADMIN])

- Verifica que \$_SESSION['user'] exista Y que rol_id esté en el array.
- Si no está autenticado: redirige al login.
- Si no tiene el rol requerido: HTTP 403 "No autorizado".

require_role([ROL_ADMIN, ROL_CAPTURISTA])

- Permite acceso a los dos roles indicados.



CONSTANTES DE ROLES (definidas en helpers.php):

```
define('ROL_ADMIN', 1);  
  
define('ROL_CAPTURISTA', 2);  
  
define('ROL_CONSULTOR', 3);
```

Ejemplo de uso en controlador:

```
private function guard(){  
  
    require_role([ROL_ADMIN, ROL_CAPTURISTA]);  
  
}
```

7. SISTEMA DE ROLES Y PERMISOS

Los roles se almacenan en la tabla 'roles' de la base de datos. El rol del usuario se almacena en usuarios.rol_id (FK a roles.id).

ID	Nombre	Accesos
1	Administrador	Todos los módulos, CRUD completo, gestión de usuarios
2	Capturista	Crear y editar ECA, inventario, y generar PDFs

Tabla 'roles' en BD:



```
CREATE TABLE roles (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(50) NOT NULL  
);
```

Los controladores que son exclusivos del administrador usan guardAdmin()

que verifica explícitamente rol_id = 1:

```
if ((int)$_SESSION['user']['rol_id'] !== 1){  
    http_response_code(403); exit('No autorizado');  
}
```

8. PROTECCIÓN CSRF

El sistema implementa protección CSRF (Cross-Site Request Forgery) en todos los formularios que modifican datos (POST).

GENERACIÓN DEL TOKEN (helpers.php — función csrf_token()):

- Si no existe `\$\_SESSION['csrf\_token']`, genera uno con `bin2hex(random_bytes(32))`.
- Devuelve el token como string hexadecimal de 64 caracteres.

USO EN VISTAS:

En cada formulario POST se incluye:



```
<input type="hidden" name="csrf_token" value="<?= csrf_token() ?>">
```

VALIDACIÓN (helpers.php — función csrf_validate()):

- Obtiene el token del POST (\$_POST['csrf_token']).
- Compara con el token guardado en sesión usando hash_equals().
- Si no coinciden: HTTP 403 y detiene la ejecución.

El uso de hash_equals() previene ataques de timing comparison.

Se llama csrf_validate() al inicio de TODOS los métodos store/update/destroy.

9. CAPA DE ACCESO A DATOS (MODELOS)

9.1 Clase DB (config/database.php)

Singleton que provee una única instancia PDO reutilizable.

Se accede con: DB::conn()

```
$stmt = DB::conn()->prepare("SELECT * FROM tabla WHERE id = ?");
```

```
$stmt->execute([$id]);
```

```
$fila = $stmt->fetch(); // array asociativo
```

Todas las consultas usan prepared statements parametrizados para prevenir inyección SQL.



9.2 Modelo User (models/User.php)

```
User.php x
models > User.php > PHP Intelephense > User > find
1  <?php
2  9 references | 0 implementations
3  class User {
4      3 references | 0 overrides
5      public static function findByEmail($email){
6          $st = DB::conn()->prepare("SELECT * FROM usuarios WHERE email = ?");
7          $st->execute([$email]);
8          return $st->fetch();
9      }
10     1 reference | 0 overrides
11     public static function all(){
12         $sql = "SELECT u.*, r.nombre AS rol FROM usuarios u JOIN roles r ON r.id=u.rol_id ORDER BY u.id DESC";
13         return DB::conn()->query($sql)->fetchAll();
14     }
15     2 references | 0 overrides
16     public static function find($id){
17         $st = DB::conn()->prepare("SELECT * FROM usuarios WHERE id=?");
18         $st->execute([$id]);
19         return $st->fetch();
20     }
21     1 reference | 0 overrides
22     public static function create($data){
23         $st = DB::conn()->prepare("INSERT INTO usuarios (nombre,email,password_hash,rol_id,activo) VALUES (?,?,,?,?)");
24         return $st->execute([
25             $data['nombre'], $data['email'],
26             password_hash($data['password'], PASSWORD_DEFAULT),
27             $data['rol_id'], isset($data['activo'])?1:0
28         ]);
29     }
30     1 reference | 0 overrides
31     public static function update($id,$data){
32         $fields = ['nombre = ?','email = ?','rol_id = ?','activo = ?'];
33         $params = [$data['nombre'],$data['email'],$data['rol_id'], isset($data['activo'])?1:0];
34         if (!empty($data['password'])){ $fields[]='password_hash = ?'; $params[] = password_hash($data['password'], PASSWORD_DEFAULT); }
35         $params[] = $id;
36         $sql = "UPDATE usuarios SET ".implode(',',$fields)." WHERE id = ?";
37         $st = DB::conn()->prepare($sql); return $st->execute($params);
38     }
39 }
```

Ilustración 6- User.php

Métodos estáticos disponibles:

User::findByEmail(\$email) Busca usuario por email. Devuelve array o null.

User::all() Lista todos los usuarios con JOIN a roles.

User::find(\$id) Busca usuario por ID. Devuelve array o null.

User::create(\$data) Inserta nuevo usuario. Hashea contraseña automáticamente con PASSWORD_DEFAULT.

User::update(\$id, \$data) Actualiza nombre, email, rol, activo. Si \$data['password'] no está vacío, también actualiza la contraseña (nuevo hash).

User::delete(\$id) Elimina al usuario por ID.

Columnas de la tabla 'usuarios':

id, nombre, email, password_hash, rol_id, activo, foto_perfil, created_at

9.3 Modelo Inventario (models/Inventario.php)

```

models > Inventario.php > ...
1  <?php
2
3  35 references | 0 implementations
4  class Inventario
5  {
6      10 references
7      private static function pdo(): PDO
8      {
9          return DB::conn();
10     }
11
12     /**
13      * LISTAR (BÚSQUEDA NORMAL)
14      *
15      * ===== */
16     1 reference | 0 overrides
17     public static function listar(string $q = ''): array
18     {
19         $pdo = self::pdo();
20
21         if ($q !== '') {
22             $st = $pdo->prepare("
23             SELECT r.*, c.nombre AS categoria, u.nombre AS unidad,
24             o.nombre AS organismo, m.nombre AS municipio
25             FROM recursos r
26             LEFT JOIN categorias c ON c.id = r.categoria_id
27             LEFT JOIN unidades u ON u.id = r.unidad_id
28             LEFT JOIN organismos o ON o.id = r.organismo_id
29             LEFT JOIN municipios m ON m.id = r.municipio_id
30             WHERE r.nombre LIKE :q OR r.clave LIKE :q
31             ORDER BY r.nombre
32             ");
33             $st->execute([':q' => "%$q%"]);
34             return $st->fetchAll(PDO::FETCH_ASSOC);
35         }
36
37         return $pdo->query("
38         SELECT r.*, c.nombre AS categoria, u.nombre AS unidad,
39         o.nombre AS organismo, m.nombre AS municipio
40         FROM recursos r
41         LEFT JOIN categorias c ON c.id = r.categoria_id
42         LEFT JOIN unidades u ON u.id = r.unidad_id
43         LEFT JOIN organismos o ON o.id = r.organismo_id
44         LEFT JOIN municipios m ON m.id = r.municipio_id
45         ORDER BY r.nombre
46         ");
47     }
48 }

```

Ilustración 7- Inventario.php

Métodos:

Inventario::listar(\$q) Lista bienes. Acepta término de búsqueda opcional por clave o nombre.

Inventario::buscar(\$id) Busca un bien por ID.



Inventario::crear(\$data) Alta de un nuevo bien (todos los campos del formulario en \$_POST).

Inventario::actualizar(\$id,\$d) Actualiza un bien existente.

Inventario::eliminar(\$id) Elimina un bien.

Tabla asociada: 'recursos' (inventario de bienes).

9.4 Modelo EcaFicha (models/EcaFicha.php)

```
<?php

require_once dirname(__DIR__) . "/config/database.php";

13 references | 0 implementations
class EcaFicha
{
    3 references
    private static function pdo(): PDO
    {
        return DB::conn();
    }

    1 reference | 0 overrides
    public static function crear(array $p): int
    {
        $pdo = self::pdo();

        $cols = $pdo->query("SHOW COLUMNS FROM eca_fichas")->fetchAll(PDO::FETCH_COLUMN);

        $insertCols = [];
        $insertVals = [];
        $params      = [];

        foreach ($cols as $col) {
            if ($col === 'id') continue;
            if ($col === 'fecha_captura') continue;

            $insertCols[] = $col;
            $insertVals[] = ":" . $col;
            $params[":" . $col] = $p[$col] ?? null;
        }

        $sql = "INSERT INTO eca_fichas (" . implode(",", $insertCols) . ")
```

Ilustración 8- EcaFicha.php

Gestiona las fichas técnicas completas de los Espacios de Cultura del Agua.

Métodos principales:

EcaFicha::todas(\$municipio_id) Lista fichas, opcionalmente filtradas.

EcaFicha::buscar(\$id) Obtiene una ficha completa por ID.

EcaFicha::crear(\$data) Inserta nueva ficha con todos los campos de las 8 secciones del formulario.

EcaFicha::actualizar(\$id,\$d) Actualiza una ficha existente.

EcaFicha::eliminar(\$id) Elimina una ficha.

Tabla asociada: 'eca_fichas' (ficha técnica de cada ECA).

9.5 Modelo Catalogo (models/Catalogo.php)

```

Catalogo.php
1  <?php
2
3  19 references | 0 implementations
4  class Catalogo
5  {
6      7 references
7      private static function pdo(): PDO
8      {
9          return DB::conn();
10     }
11
12     2 references | 0 overrides
13     public static function categorias(): array
14     {
15         return self::pdo()
16             ->query("SELECT id, nombre FROM categorias ORDER BY nombre")
17             ->fetchAll(PDO::FETCH_ASSOC);
18     }
19
20     1 reference | 0 overrides
21     public static function unidades(): array
22     {
23         return self::pdo()
24             ->query("SELECT id, nombre FROM unidades ORDER BY nombre")
25             ->fetchAll(PDO::FETCH_ASSOC);
26     }
27
28     3 references | 0 overrides
29     public static function organismos(): array
30     {
31         return self::pdo()
32             ->query("SELECT id, nombre, municipio_id FROM organismos ORDER BY nombre")
33             ->fetchAll(PDO::FETCH_ASSOC);
34     }
35
36     3 references | 0 overrides
37     public static function municipios(): array

```

Ilustración 9- Catalogo.php

Provee listas de opciones para los formularios (todos los métodos devuelven arrays para poblar <select>).

<i>Catalogo::categorias()</i>	Lista de categorías de bienes.
<i>Catalogo::unidades()</i>	Unidades de medida (Pieza, Juego, etc.).
<i>Catalogo::organismos()</i>	Organismos operadores.
<i>Catalogo::municipios()</i>	Municipios del estado.
<i>Catalogo::proveedores()</i>	Proveedores registrados.
<i>Catalogo::ubicaciones()</i>	Ubicaciones físicas.

9.6 Otros modelos

<i>Role::all()</i>	Lista todos los roles disponibles.
<i>Role::find(\$id)</i>	Busca un rol por ID. Usado en validación de usuarios.
<i>Municipio::all()</i>	Lista municipios (id, nombre).
<i>Organismo::all()</i>	Lista organismos (id, nombre, municipio_id).
<i>Organismo::porMunicipio(\$mun_id)</i>	Filtra organismos por municipio.
<i>Reporte::todos()</i>	Lista reportes (municipales y anuales).
<i>Reporte::eliminar(\$id)</i>	Elimina un reporte y su archivo PDF del disco.



10. CONTROLADORES

10.1 AuthController

```
AuthController.php M X
controllers > AuthController.php > ...
 2  class AuthController {
16
17  1 reference
17  private function render($view, $params=[]){
18      extract($params);
19      ob_start();
20      require __DIR__."/../views/$view.php";
21      return ob_get_clean();
22  }
23
24  0 references | 0 overrides
24  public function login(){
25      if (empty($_SESSION['user'])) { header('Location: index.php?controller=dashboard&action=inicio'); exit; }
26      return $this->render('auth/login');
27  }
28
29  0 references | 0 overrides
29  public function doLogin(){
30      $email = trim($_POST['email'] ?? '');
31      $pass  = $_POST['password'] ?? '';
32
33      $user = User::findByEmail($email);
34
35      // Verificar credenciales - log si falla
36      if (!($user || password_verify($pass, $user['password_hash'] ?? '') || !($user['activo'] ?? 0))) {
37          Bitacora::registrar('login_fallido', 'auth', 'Intento de acceso fallido', [
38              'email_intento' => $email,
39              'ip'            => $_SERVER['REMOTE_ADDR'] ?? null,
40          ]);
41          $_SESSION['flash'] = 'Credenciales inválidas';
42          header('Location: index.php?controller=auth&action=login');
43          exit;
44      }
45
46      // Login exitoso
47      $_SESSION['user'] = [
48          'id' => $user['id'],
```

Ilustración 10- Controlador

Archivo: controllers/AuthController.php

Acciones:

login() GET → Muestra el formulario de login. Si ya hay sesión activa, redirige al dashboard.

doLogin() POST → Valida credenciales, abre sesión, registra en bitácora.

logout() GET → Destruye la sesión, registra en bitácora, redirige.

10.2 DashboardController

Archivo: controllers/DashboardController.php

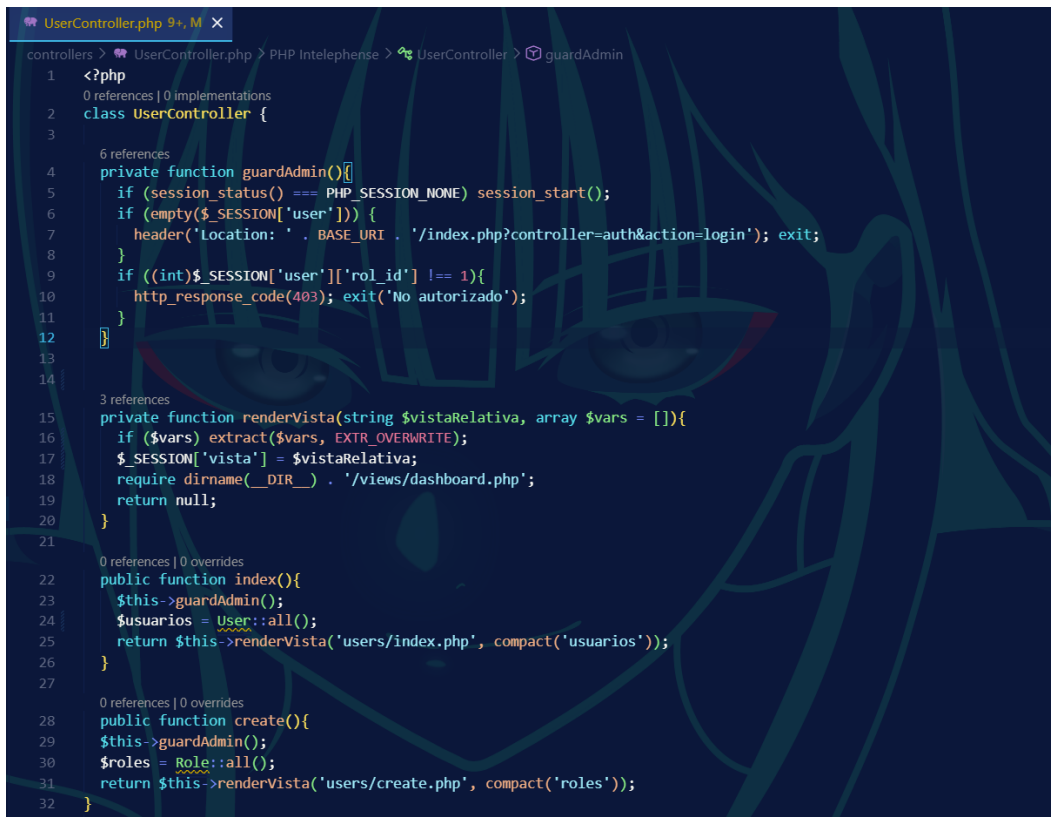
Acciones:

inicio() GET → Muestra la pantalla de bienvenida.

perfil() GET → Muestra el formulario de perfil (datos + contraseña).

(Los cambios de perfil y contraseña se manejan desde archivos separados en controllers/actualizar_perfil.php y controllers/cambiar_password.php, invocados vía petición POST desde la vista.)

10.3 UserController



```

UserController.php 9+, M X
controllers > UserController.php > PHP Intelephense > UserController > guardAdmin
1  <?php
  0 references | 0 implementations
2  class UserController {
3
4      6 references
5      private function guardAdmin(){
6          if (session_status() === PHP_SESSION_NONE) session_start();
7          if (empty($_SESSION['user'])) {
8              header('Location: ' . BASE_URI . '/index.php?controller=auth&action=login'); exit;
9          }
10         if ((int)$_SESSION['user']['rol_id'] !== 1){
11             http_response_code(403); exit('No autorizado');
12         }
13     }
14
15     3 references
16     private function renderVista(string $vistaRelativa, array $vars = []){
17         if ($vars) extract($vars, EXTR_OVERWRITE);
18         $_SESSION['vista'] = $vistaRelativa;
19         require dirname(__DIR__) . '/views/dashboard.php';
20         return null;
21     }
22
23     0 references | 0 overrides
24     public function index(){
25         $this->guardAdmin();
26         $usuarios = User::all();
27         return $this->renderVista('users/index.php', compact('usuarios'));
28     }
29
30     0 references | 0 overrides
31     public function create(){
32         $this->guardAdmin();
33         $roles = Role::all();
34         return $this->renderVista('users/create.php', compact('roles'));
35     }
36 }
  
```

Ilustración 11- UserController.php



Archivo: controllers/UserController.php

Protección: Solo rol_id = 1 (Administrador)

Acciones:

index() GET → Lista todos los usuarios.

create() GET → Muestra formulario de nuevo usuario.

store() POST → Valida y guarda nuevo usuario. Registra en bitácora.

edit() GET → Muestra formulario de edición (precargado).

update() POST → Valida y actualiza usuario. Registra en bitácora.

destroy() POST → Elimina usuario por ID. Registra en bitácora.

Validaciones implementadas:

- nombre: mínimo 3 caracteres.
- email: formato válido con filter_var(FILTER_VALIDATE_EMAIL).
- email: único en BD (con excepción del propio usuario en edición).
- password: mínimo 8 caracteres.
- password_confirm: debe coincidir con password.
- rol_id: debe existir en la tabla roles.



10.4 FormatoController

```
1  <?php
2  use PhpOffice\PhpSpreadsheet\Spreadsheet;
3  use PhpOffice\PhpSpreadsheet\IOFactory;
4  use PhpOffice\PhpSpreadsheet\Cell\Coordinate;
5
6  0 references | 0 implementations
7  class FormatoController
8  {
9
10     12 references
11     private function guard(): void
12     {
13         require_role([ROL_ADMIN, ROL_CAPTURISTA]);
14     }
15
16     4 references
17     private function render(string $vista, array $vars = []): void
18     {
19         $this->guard();
20
21         extract($vars);
22
23         $_SESSION['vista'] = "formatos/{$vista}.php";
24         require_dirname(__DIR__) . '/views/dashboard.php';
25     }
26
27     0 references | 0 overrides
28     public function index(): void
29     {
30         $formatos = Formato::all();
31         $municipios = Municipio::listar();
32         $organismos = Organismo::listar();
33
34         $pdo = DB::conn();
```

Ilustración 12- Formato Cotroller

Archivo: controllers/FormatoController.php

Acciones:

index() GET → Dashboard de formatos con datos para gráficas Chart.js.

capturaECA() GET → Formulario de nueva ficha ECA.

storeECA() POST → Guarda nueva ficha ECA.

consultaECA() GET → Lista/busca fichas con filtro por municipio.



- verECA() GET → Muestra detalle de una ficha (solo lectura).
- editarECA() GET → Formulario de edición prellenado.
- updateECA() POST → Actualiza ficha ECA existente.
- reporte() GET → Muestra el generador de reportes PDF.
- generarPDF() POST → Genera y descarga el PDF según tipo (municipal/anual).

10.5 InventarioController

```
<?php
0 references | 0 implementations
class InventarioController
{
    3 references
    private function json(array $payload, int $statusCode = 200): void
    {
        http_response_code($statusCode);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode($payload);
        exit;
    }

    6 references
    private function guard()
    {
        require_role([ROL_ADMIN, ROL_CAPTURISTA]);
    }

    2 references
    private function render(string $vista, array $vars = [])
    {
        extract($vars);
        $_SESSION['vista'] = $vista;
        require dirname(__DIR__) . '/views/dashboard.php';
    }

    0 references | 0 overrides
    public function index()
    {
        $this->guard();

        $q = $_GET['q'] ?? '';
        $recursos = Inventario::listar($q);

        $this->render('inventario/index.php', compact('recursos', 'q'));
```

Ilustración 13- Inventario Controller

Archivo: controllers/InventarioController.php

Protección: ROL_ADMIN o ROL_CAPTURISTA

Acciones:

index() GET → Lista bienes con búsqueda por texto.

form() GET → Formulario de alta/edición (carga catálogos de BD).

store() POST → Valida y guarda nuevo bien. Registra en bitácora.

update() POST → Actualiza bien existente. Registra en bitácora.

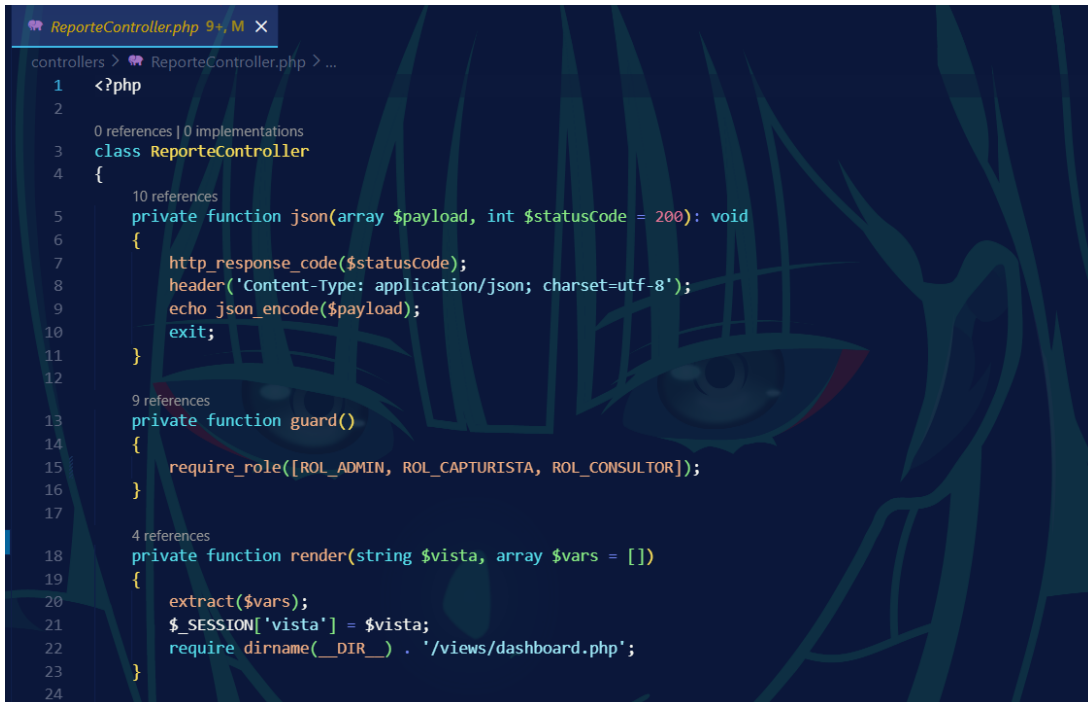
delete() GET → Elimina bien por ID. Registra en bitácora.

edit() GET → Devuelve JSON con datos del bien (para modal AJAX).

Respuestas JSON (para AJAX):

edit() devuelve Content-Type: application/json con el registro del bien o {"error": "Recurso no encontrado"} con HTTP 404.

10.6 ReporteController



```
1 <?php
2
3 0 references | 0 implementations
4 class ReporteController
5 {
6     10 references
7     private function json(array $payload, int $statusCode = 200): void
8     {
9         http_response_code($statusCode);
10        header('Content-Type: application/json; charset=utf-8');
11        echo json_encode($payload);
12        exit;
13    }
14
15    9 references
16    private function guard()
17    {
18        require_role([ROL_ADMIN, ROL_CAPTURISTA, ROL_CONSULTOR]);
19    }
20
21    4 references
22    private function render(string $vista, array $vars = [])
23    {
24        extract($vars);
25        $_SESSION['vista'] = $vista;
26        require dirname(__DIR__) . '/views/dashboard.php';
27    }
28 }
```

Ilustración 14- Reporte controller

Archivo: controllers/ReporteController.php

Acciones:

municipales() GET → Lista reportes municipales con filtros opcionales.

anuales() GET → Lista reportes anuales con filtros por año/fecha.

inventario() GET → Vista de inventario con búsqueda, filtros y paginación.

eliminar() POST → Elimina un reporte y su archivo PDF. CSRF protegido.

Registra en bitácora.

10.7 ManualController

```
<?php
0 references | 0 implementations
class ManualController
{
    1 reference
    private function findManualPath(): ?string
    {
        $preferred = [
            dirname(__DIR__) . '/public/manuales/manual_usuario_ceaa.pdf',
            dirname(__DIR__) . '/public/manuales/manual_tecnico_ceaa2.pdf',
            dirname(__DIR__) . '/public/manuales/manual_tecnico_ceaa.pdf',
            dirname(__DIR__) . '/public/manuales/Manual_Tecnico_CEEA.pdf',
            dirname(__DIR__) . '/public/manuales/manual-tecnico-ceaa.pdf',
        ];

        foreach ($preferred as $path) {
            if (is_file($path)) {
                return $path;
            }
        }

        $manualesDir = dirname(__DIR__) . '/public/manuales';
        if (is_dir($manualesDir)) {
            $pdfs = glob($manualesDir . '/*.pdf') ?: [];
            if (!empty($pdfs)) {
                usort($pdfs, fn($a, $b) => filemtime($b) <=> filemtime($a));
                return $pdfs[0];
            }
        }

        $legacyDir = dirname(__DIR__) . '/public/pdf';
        if (is_dir($legacyDir)) {
            $manuallike = glob($legacyDir . '/*{manual,Manual,tecnico,Tecnico,guia,Guia}*.pdf', GLOB_BRACE) ?: [];
            if (!empty($manuallike)) {
                usort($manuallike, fn($a, $b) => filemtime($b) <=> filemtime($a));
            }
        }
    }
}
```

Ilustración 15- Manual controller



Archivo: controllers/ManualController.php

Acciones:

ver() GET → Sirve el archivo PDF técnico con headers apropiados (Content-Type: application/pdf). El archivo se lee desde public/manuales/ con readfile().

10.8 ChatbotController

```
<?php

0 references | 0 implementations
class ChatbotController
{
    /**
     * POST /index.php?controller=chatbot&action=rating
     * Body JSON: { intent: string, vote: 0|1, _csrf_token: string }
     * Guarda la calificación en scripts/chatbot_ratings.json
     */
    0 references | 0 overrides
    public function rating(): string
    {
        if (empty($_SESSION['user'])) {
            http_response_code(401);
            header('Content-Type: application/json; charset=utf-8');
            echo json_encode(['ok' => false, 'error' => 'No autenticado']);
            exit;
        }

        if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
            http_response_code(405);
            header('Content-Type: application/json; charset=utf-8');
            echo json_encode(['ok' => false, 'error' => 'Método no permitido']);
            exit;
        }

        $raw = file_get_contents('php://input');
        $data = json_decode($raw, true);

        if (!is_array($data)) {
            http_response_code(400);
            header('Content-Type: application/json; charset=utf-8');
            echo json_encode(['ok' => false, 'error' => 'JSON inválido']);
            exit;
        }
    }
}
```

Ilustración 16- ChatBot Controller

Archivo: controllers/ChatbotController.php

Acciones:



responder() POST → Recibe mensaje del usuario, devuelve respuesta JSON.

rating() POST → Guarda la calificación del chatbot.

inventarioEstados() GET → Devuelve JSON con estados del inventario (para consultas del asistente).

11. VISTAS Y LAYOUT

11.1 Plantilla maestra (views/dashboard.php)

```
views > dashboard.php > html > body > div#wrapper.wrapper > aside#sidebar.barra-lateral > div.user-info
1 <?php
2 if (session_status() === PHP_SESSION_NONE) session_start();
3 if (!isset($_SESSION['user'])) {
4     header('Location: ' . BASE_URI . '/index.php?controller=auth&action=login');
5     exit;
6 }
7
8 $usuario = $_SESSION['user'];
9 $currentPage = $_GET['action'] ?? '';
10 $currentCtrl = $_GET['controller'] ?? '';
11 ?>
12 <!doctype html>
13 <html lang="es">
14 <head>
15 <meta charset="utf-8">
16 <title>Panel de Administración - CEEA</title>
17 <meta name="viewport" content="width=device-width, initial-scale=1">
18 <link rel="icon" type="image/png" href="<?= asset('img/logoo.png') ?>">
19 <link rel="stylesheet" href="<?= asset('css/dashboard.css') ?>?v=3">
20 <link rel="stylesheet" href="<?= asset('css/dashboard.css') ?>?v=3">
21 <link rel="stylesheet" href="<?= asset('css/chatbot-soporte.css') ?>?v=2">
22 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.5.0/css/all.min.css">
23
24 <?php
25 $currentCtrl = $_GET['controller'] ?? '';
26
27 // Formatos
28 if ($currentCtrl === 'formatos'): ?>
29     <link rel="stylesheet" href="<?= asset('css/formatos.css') ?>?v=2">
30 <?php endif; ?>
31
32 <?php
33 // Inventario (captura) y Reportes de inventario
34 if ($currentCtrl === 'inventario' || ($currentCtrl === 'reportes' && $currentPage === 'inventario')): ?>
35     <link rel="stylesheet" href="<?= asset('css/inventario-reporte.css') ?>?v=3">
36 <?php endif; ?>
37 <?php
```

Ilustración 17- Pantilla maestra(*dashboard.php*)

Todas las páginas internas del sistema usan esta plantilla como layout. Los controladores la incluyen de esta forma:



```
$_SESSION['vista'] = 'formatos/eca_consulta.php';
```

```
require dirname(__DIR__) . '/views/dashboard.php';
```

La plantilla:

1. Verifica autenticación (si no hay sesión, redirige al login).
2. Lee la foto de perfil actualizada directamente desde la BD.
3. Renderiza el <head> con metadatos y CSS base.
4. Carga selectivamente CSS adicionales según el controlador activo (ver sección 11.3).
5. Renderiza el Header con el logo institucional.
6. Renderiza el Sidebar con datos del usuario y menú de navegación.
7. Incluye la vista definida en \$_SESSION['vista'] dentro del contenedor principal.
8. Incluye el widget del chatbot (botón flotante + ventana).

11.2 Organización de vistas

views/auth/ Vistas del módulo de autenticación

views/dashboard/ Inicio y perfil

views/formatos/ Todas las vistas relacionadas con fichas ECA y reportes

views/inventario/ Alta y formulario de bienes

views/reportes/ Listados de reportes generados e inventario

views/users/ Gestión de usuarios



views/layouts/ Fragmentos HTML reutilizables (si existen)

11.3 Carga condicional de hojas de estilo

El layout dashboard.php solo carga los CSS necesarios para cada módulo:

controller=formatos

→ public/assets/css/formatos.css

controller=inventario

→ public/assets/css/inventario-captura.css

→ public/assets/css/inventario-reporte.css

controller=reportes, action=inventario

→ public/assets/css/inventario-reporte.css

→ public/assets/css/excel-modal.css

controller=formatos, action=capturaECA

→ public/assets/css/captura-eca.css

Siempre:

→ public/assets/css/dashboard.css (estilos base + sidebar + header)

→ public/assets/css/chatbot-soporte.css

→ Font Awesome 6.5.0 (CDN)

Para forzar recarga del caché en desarrollo, los CSS se cargan con parámetros de versión: css/dashboard.css?v=3.

12. GENERACIÓN DE PDFs

El sistema usa dos librerías distintas para generar PDFs:

- mPDF (mpdf/mpdf) → Reportes de inventario municipales y anuales.
- dompdf (dompdf/dompdf) → Fichas técnicas ECA.

Ambas se instalan vía Composer y están en el directorio vendor/.

12.1 Reportes municipales

(controllers/export/InventarioFormatoMunicipioPDF.php)

```
InventarioFormatoMunicipio.php X
controllers > export > InventarioFormatoMunicipio.php > ...
1  <?php
2  require_once dirname(__DIR__) . '/config/config.php';
3  require_once dirname(__DIR__) . '/models/Inventario.php';
4  require_once dirname(__DIR__) . '/vendor/autoload.php';
5
6  use PhpOffice\PhpSpreadsheet\IOFactory;
7  use PhpOffice\PhpSpreadsheet\Worksheet\Drawing;
8  use PhpOffice\PhpSpreadsheet\Style\Border;
9
10 $anio = $_GET['anio'] ?? date('Y');
11 $municipioId = $_GET['municipio_id'] ?? null;
12 $organismoId = $_GET['organismo_id'] ?? null;
13
14 $template = __DIR__ . '/Formatos.xlsx';
15 $excel = IOFactory::load($template);
16
17 // HOJA
18 $hoja = $excel->getSheetByName("Formato por municipio");
19
20 // LOGO
21 $logo = new Drawing();
22 $logo->setName("Logo CEAA");
23 $logo->setPath(dirname(__DIR__) . '/public/assets/img/logo.png');
24 $logo->setCoordinates('A1');
25 $logo->setHeight(85);
26 $logo->setOffsetX(5);
27 $logo->setWorksheet($hoja);
28
29 // ENCABEZADO
30 $nombreMunicipio = Inventario::obtenerNombreMunicipio($municipioId);
31 $hoja->mergeCells("A5:O5");
32 $hoja->setCellValue("A5", "Fortalecimiento de Espacios de Cultura del Agua del Municipio de $nombreMunicipio");
33 $hoja->getStyle("A5")->getFont()->setBold(true)->setSize(16);
34 $hoja->getStyle("A5")->getAlignment()->setHorizontal('center');
35
36 // DATOS
```

Ilustración 18- InventarioFormatoMunicipal.php

Flujo:

1. FormatoController recibe via POST: municipio_id, organismo_id, beneficiario, accion, anio, y los IDs de recursos seleccionados.

2. Consulta los recursos en BD.

3. Genera HTML con la tabla de bienes.

4. Crea instancia de \Mpdf\Mpdf y escribe el HTML.

5. Devuelve el PDF como descarga (Content-Disposition: attachment).

El PDF se guarda en: public/pdf/ con nombre generado dinámicamente.

El registro del archivo se guarda en la tabla 'reportes' de la BD.

12.2 Reportes anuales (controllers/export/InventarioFormatoAnualPDF.php)

```

1  <?php
2  require __DIR__ . "/../../vendor/autoload.php";
3
4  use Dompdf\Dompdf;
5  use Dompdf\Options;
6
7  $options = new Options();
8  $options->set('isRemoteEnabled', true);
9
10 $dompdf = new Dompdf($options);
11
12 $data = $reporteData ?? [];
13 $anio = $_GET['anio'] ?? date("Y");
14
15 ob_start();
16 ?>
17
18 <style>
19 body { font-family: DejaVu Sans, sans-serif; font-size: 11px; }
20 h2 { text-align:center; margin-bottom: 10px; }
21 table { width:100%; border-collapse: collapse; font-size: 10px; }
22 th, td { border:1px solid #000; padding:4px; }
23 th { background:#e8e8e8; }
24 .header-logos { width: 100%; text-align:center; }
25 </style>
26
27 <div class="header-logos">
28   
29 </div>
30
31 <h2>Fortalecimiento de Espacios de Cultura del Agua - Consolidado <?= $anio ?></h2>
32
33 <table>
34 <thead>
35 <tr>
36   <th>No.</th>
37   <th>Municipios</th>

```

Ilustración 19- InventarioFormatoAnual.php



Similar al municipal, pero consolida todos los recursos del año indicado, sin filtro de municipio/organismo.

12.3 Ficha técnica ECA (controllers/export/pdf_eca.php)

```
1 <?php
2 require_once dirname(__DIR__, 2) . '/vendor/autoload.php';
3
4 1 reference
5 ✓ function generarPDF_ECA(array $ficha)
6 {
7     $mpdf = new \Mpdf\Mpdf([
8         'margin_top' => 10,
9         'margin_bottom' => 10,
10        'margin_left' => 10,
11        'margin_right' => 10,
12        'default_font_size' => 10,
13        'default_font' => 'Arial'
14    ]);
15    /* ----- LOGO ----- */
16    $logo = dirname(__DIR__, 2) . "/public/assets/img/Logotipo1.png";
17    $logo = str_replace("\\", "/", $logo);
18
19    /* ----- CSS ----- */
20    $css = "
21        body { font-family: Arial, sans-serif; }
22
23        table { width: 100%; border-collapse: collapse; }
24
25        td, th {
26            border: 1px solid #333;
27            padding: 5px;
28            font-size: 11px;
29        }
30
31        .titulo-principal {
32            background: #7a0d1c;
33            color: white;
34            font-size: 20px;
35            font-weight: bold;
36            padding: 10px;
```

Ilustración 20- pdf_eca.php

Flujo:

1. Recibe el ID de la ficha ECA via GET.
2. Carga todos los datos de la ficha desde EcaFicha::buscar(\$id).
3. Construye HTML estructurado con todas las 8 secciones.
4. Genera el PDF con dompdf.



5. Devuelve descarga directa.

13. EXPORTACIÓN A EXCEL (PhpSpreadsheet)

Disponible para el inventario.

Archivos:

controllers/export/InventarioFormatoMunicipio.php → Excel municipal

controllers/export/InventarioFormatoAnual.php → Excel anual global

Flujo general:

1. Se instancia \PhpOffice\PhpSpreadsheet\Spreadsheet.
2. Se construye la hoja con encabezados y filas de datos.
3. Se escribe via \PhpOffice\PhpSpreadsheet\Writer\Xlsx.
4. Se devuelve con Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet y Content-Disposition: attachment.



14. SISTEMA DE BITÁCORA (AUDITORÍA)

14.1 Clase Bitacora (models/Bitacora.php)

```
<?php
16 references | 0 implementations
class Bitacora
{
    16 references | 0 overrides
    public static function registrar(string $accion, string $modulo, string $detalle = '', array $metadata = []): void
    {
        try {
            $userId = null;
            if (!empty($_SESSION['user']['id'])) {
                $userId = (int)$_SESSION['user']['id'];
            }

            $ip = $_SERVER['REMOTE_ADDR'] ?? null;
            $userAgent = $_SERVER['HTTP_USER_AGENT'] ?? null;
            $metadataJson = !empty($metadata)
                ? json_encode($metadata, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES)
                : null;

            $sql = "
                INSERT INTO bitacora_acciones
                (user_id, accion, modulo, detalle, ip, user_agent, metadata_json, creado_en)
                VALUES
                (?, ?, ?, ?, ?, ?, ?, NOW())
            ";

            $stmt = DB::conn()->prepare($sql);
            $stmt->execute([
                $userId,
                $accion,
                $modulo,
                $detalle,
                $ip,
                $userAgent,
                $metadataJson,
            ]);
        }
    }
}
```

Ilustración 21- Bitacora.php

Método único:

Bitacora::registrar(string \$accion, string \$modulo, string \$detalle = "", array \$metadata = [])

Parámetros:

\$accion Tipo de acción: 'login', 'logout', 'login_fallido', 'crear', 'actualizar', 'eliminar', etc.

\$modulo Módulo donde ocurrió: 'auth', 'usuarios', 'inventario', 'formatos', 'reportes', etc.

\$detalle Descripción legible de la acción.

\$metadata Array con datos extra que se serializa a JSON.



Captura automáticamente:

- user_id: desde \$_SESSION['user']['id']
- ip: desde \$_SERVER['REMOTE_ADDR']
- user_agent: desde \$_SERVER['HTTP_USER_AGENT']
- creado_en: timestamp con NOW()

Los errores se silencian (catch vacío) para no interrumpir el flujo de la aplicación si la bitácora falla.

14.2 Tabla bitacora_acciones

CREATE TABLE bitacora_acciones (

id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,

user_id INT DEFAULT NULL,

accion VARCHAR(100) NOT NULL,

modulo VARCHAR(100) NOT NULL,

detalle TEXT,

ip VARCHAR(45) DEFAULT NULL,

user_agent VARCHAR(500) DEFAULT NULL,

metadata_json TEXT DEFAULT NULL,

creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

PRIMARY KEY (id),

INDEX idx_user_id (user_id),

INDEX idx_modulo (modulo),

INDEX idx_creado_en (creado_en)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

NOTA: ip VARCHAR(45) soporta tanto IPv4 como IPv6.

14.3 Eventos registrados por módulo

MÓDULO	ACCIÓN	CUANDO OCURRE
Auth	Login	Inicio de sesión exitoso
Auth	Login_fallido	Credenciales incorrectas o usuario inactivo
Auth	logout	Cierre de sesión explícito
Usuarios	Crear	Nuevo usuario creado
Usuarios	Actualizar	Datos de usuario modificados
Usuarios	eliminar	Usuario eliminado
Inventario	Crear	Nuevo bien registrado
Inventario	Actualizar	Bien modificado
Inventario	Eliminar	Bien eliminado
Formatos	Crear_eca	Nueva ficha ECA guardada
Formatos	Actualizar_eca	Ficha ECA modificada
Formatos	Generar_pdf	PDF generado y descargado
Reportes	Eliminar	Reporte PDF eliminado



15. SUBIDA DE ARCHIVOS

El sistema gestiona dos tipos de archivos subidos:

A) FOTOS DE PERFIL

- Campo HTML : `<input type="file" name="foto_perfil">`
- Válidos : JPG, JPEG, PNG (validado por `$_FILES['type']` y extensión)
- Destino : `public/assets/uploads/perfiles/`
- Nombre archivo : `perfil_{user_id}_{timestamp}.{ext}`
- Se actualiza en BD : `UPDATE usuarios SET foto_perfil = ? WHERE id = ?`
- El layout (`dashboard.php`) lee directamente desde BD para evitar inconsistencias de caché de sesión.

B) DOCUMENTOS DE INVENTARIO

- Campo HTML : `<input type="file" name="documento">`
- Válidos : PDF, JPG, PNG
- Destino : `public/uploads/` o subcarpeta configurada
- Nombre archivo : `doc_{recurso_id}_{timestamp}.{ext}`
- Se almacena la ruta relativa en la tabla recursos.

RECOMENDACIÓN DE SEGURIDAD:

- Validar siempre tanto el MIME type real (`finfo_file`) como la extensión.
- Nunca ejecutar archivos subidos — almacenarlos en un directorio sin

permisos de ejecución del servidor web.

- Considerar mover el directorio de uploads fuera del webroot en producción.

16. ESQUEMA DE BASE DE DATOS

16.1 Tablas principales

TABLA	DESCRIPCIÓN
Roles	Catálogo de roles (id, nombre)
Usuarios	Cuentas de acceso (id, nombre, email, password_hash, rol_id FK, activo, foto_perfil, created_at)
Municipios	Municipios del estado (id, nombre, clave)
Organismos	Organismos operadores (id, nombre, municipio_id FK)
Eca_fichas	Fichas técnicas ECA — tabla central del sistema. Relacionada con municipios y organismos.
Recursos	Inventario de bienes (clave, nombre, categoría, estado, municipio_id, organismo_id, costo, marca, serie, etc.)
Categorías	Catálogo de categorías de bienes
Unidades_medida	Catálogo de unidades de medida
Proveedores	Catálogo de proveedores
Reportes	Registro de PDFs generados (tipo, archivo, municipio_id, organismo_id, anio, created_at)
Bitacora_acciones	Auditoría de todas las acciones del sistema

16.2 Relaciones clave

usuarios.rol_id → roles.id

organismos.municipio_id → municipios.id



eca_fichas.municipio_id → municipios.id

eca_fichas.organismo_id → organismos.id

recursos.municipio_id → municipios.id

recursos.organismo_id → organismos.id

bitacora_acciones.user_id → usuarios.id (sin FK formal — registro permisivo)

Los scripts de actualización de esquema se encuentran en sql_updates/:

- agregar_cambios_inventario.sql : Columnas adicionales para recursos
- 2026-03-02_operativa_*.sql : Índices para bitácora
- Ver sql_updates/README.md : Historial de cambios

Para aplicar una actualización:

```
mysql -u root sistema_ceaa < sql_updates/nombre_del_script.sql
```

17. DEPENDENCIAS EXTERNAS (COMPOSER)

Archivo: composer.json

```
{  
  
  "require": {  
  
    "phpoffice/phpspreadsheet": "^5.2",  
  
    "dompdf/dompdf": "3.1",  
  
    "mpdf/mpdf": "8.2"
```



}

}

LIBRERÍA	USO EN EL SISTEMA
phpoffice/phpspreadsheet	Generación de archivos Excel (.xlsx) para los reportes de inventario municipales y anuales.
dompdf/dompdf	Renderizado de HTML a PDF. Usado para las fichas técnicas ECA (pdf_eca.php).
mpdf/mpdf	Generación avanzada de PDF con soporte UTF-8 y tablas complejas. Usado para reportes de inventario.

INSTALAR DEPENDENCIAS (primera vez o tras clonar el proyecto):

```
cd c:\xampp\htdocs\ceaa_recursos
```

```
composer install
```

ACTUALIZAR:

```
composer update
```

El directorio vendor/ NO debe subirse al control de versiones.

Asegúrese de tener un archivo .gitignore con: vendor/



18. ESTRUCTURA DE ASSETS

public/assets/

├─ css/

| └─ dashboard.css Estilos globales: sidebar, header, layout

| └─ chatbot-soporte.css Estilos del widget chatbot

| └─ formatos.css Módulo de formatos/ECA

| └─ captura-eca.css Formulario de captura ECA (8 secciones)

| └─ inventario-captura.css Formulario de alta de bienes

| └─ inventario-reporte.css Tabla de inventario con filtros y paginación

| └─ excel-modal.css Modal de edición en inventario

├─ js/

| └─ dashboard.js Interactividad general (sidebar toggle, etc.)

| └─ formatos.js Gráficas Chart.js del módulo formatos

| └─ [otros módulos].js

├─ img/

| └─ Logotipo1.png Logo del header

| └─ logoo.png Favicon

| └─ [otras imágenes].png

├─ formatos/ Plantillas de documentos descargables

└─ uploads/perfiles/ Fotos de perfil de usuarios (generadas dinámicamente)



public/pdf/ PDFs generados por el sistema (reportes)

public/manuales/ Manual técnico en PDF

19. SCRIPTS DE MANTENIMIENTO

Los scripts de utilidades se encuentran en la carpeta scripts/:

migrar_passwords.php

Migra contraseñas en texto plano a hashes bcrypt con PASSWORD_DEFAULT.

Ejecutar UNA sola vez si se importaron usuarios con contraseñas sin hashear.

Uso: php scripts/migrar_passwords.php

seeder_recursos_equipo_2025.php / .sql

Carga masiva de bienes del inventario para el año 2025 desde el CSV.

Uso: php scripts/seeder_recursos_equipo_2025.php

seeder_log.txt / seeder_output.txt

Logs generados por el seeder para verificar resultados.

scripts/err.txt / scripts/out.txt

Salidas capturadas de ejecuciones anteriores.



20. SEGURIDAD APLICADA

MEDIDA	IMPLEMENTACIÓN
SQL Injection	Todos los queries usan PDO prepared statements con parámetros enlazados (?).
XSS	Las vistas usan htmlspecialchars() para toda salida de datos del usuario.
CSRF	Token único por sesión en todos los formularios POST. Validado con hash_equals().
Acceso no autorizado	Cada controlador llama a require_role() o guardAdmin() antes de ejecutar cualquier lógica.
Contraseñas	Almacenadas con bcrypt (PASSWORD_DEFAULT). Verificadas con password_verify(). Nunca en texto plano ni en logs.
Sesiones	La sesión se inicia con session_start() una sola vez. Se destruye con session_destroy() al cerrar sesión.
Datos de sesión	Solo se guarda id, nombre, email, rol_id y foto_perfil. Nunca la contraseña.
Auditoría	Todos los accesos (exitosos y fallidos) y cambios de datos quedan en bitacora_acciones.
Uploads	Se valida extensión y tipo de archivo antes de mover a destino.



Cache de CSS/JS	Se añaden parámetros ?v=N para forzar recarga tras actualizaciones sin exponer rutas reales.
-----------------	--

PENDIENTES RECOMENDADOS PARA PRODUCCIÓN:

- Habilitar HTTPS con certificado SSL válido.
- Cambiar credenciales de BD (usuario root sin contraseña).
- Configurar Content Security Policy (CSP) en headers HTTP.
- Habilitar rate limiting en la acción doLogin() para prevenir fuerza bruta.
- Mover la carpeta uploads fuera del webroot.
- Deshabilitar display_errors en php.ini de producción.
- Configurar un log de errores PHP en archivo (log_errors = On).

21. ERRORES COMUNES Y SOLUCIONES

PROBLEMA: "Controlador no encontrado" en la URL

CAUSA : El parámetro ?controller no coincide con ninguna entrada del mapa.

SOLUCIÓN: Verificar que el nombre en la URL coincida exactamente con las claves del mapa en public/index.php (minúsculas y sin acentos).

PROBLEMA: "No se encontró la clase 'XController'"

CAUSA : El archivo del controlador no existe o el nombre de la clase dentro del archivo no coincide exactamente con el nombre del archivo.



SOLUCIÓN: Verificar que el archivo sea controllers/NombreController.php y
que declare class NombreController { }.

PROBLEMA: El PDF descargado está en blanco o muestra errores

CAUSA : Falta de memoria PHP o conflicto de librerías PDF.

SOLUCIÓN: Aumentar memory_limit en php.ini a 256M o más. Verificar que
vendor/ esté actualizado con composer install.

PROBLEMA: Las fotos de perfil no se muestran

CAUSA : Ruta relativa de la foto almacenada en BD no coincide con la
ruta real en disco, o el directorio uploads/perfiles/ no tiene
permisos de escritura.

SOLUCIÓN: Verificar permisos del directorio (chmod 755 en Linux).

Verificar que las rutas en BD inicien sin barra: assets/uploads/...

PROBLEMA: Error "SQLSTATE[HY000]: Can't connect to MySQL server"

CAUSA : MySQL/MariaDB no está en ejecución o sus credenciales cambiaron.

SOLUCIÓN: Iniciar el servicio MySQL desde el Panel de Control de XAMPP.

Si las credenciales cambiaron, actualizar config/database.php.

PROBLEMA: Sesión expira o se pierde al navegar



CEAA
COMISIÓN ESTATAL
DEL AGUA Y
ALCANTARILLADO



CAUSA : PHP session garbage collection o configuración de php.ini.

SOLUCIÓN: Aumentar session.gc_maxlifetime en php.ini (ej. 7200 para 2 horas).

Verificar que session.save_path tenga permisos de escritura.

PROBLEMA: Error CSRF "token inválido" al enviar formulario

CAUSA : La sesión expiró entre la carga del formulario y el envío,
o el formulario no incluye el campo csrf_token.

SOLUCIÓN: Recargar la página y volver a enviar. Si ocurre frecuentemente,
aumentar el tiempo de sesión. Verificar que la vista incluya
<?= csrf_token() ?> en el campo hidden.

PROBLEMA: No aparecen registros en la bitácora

CAUSA : La tabla bitacora_acciones no fue creada en la BD.

SOLUCIÓN: Ejecutar:

```
mysql -u root sistema_ceaa < sql_updates/2026-03  
02_operativa_bitacora_indices.sql
```

O crear la tabla manualmente con el script del manual.

22. GUÍA DE DESPLIEGUE Y ACTUALIZACIÓN

INSTALACIÓN INICIAL

1. Copiar el proyecto al directorio del servidor web:



C:\xampp\htdocs\ceaa_recursos\ (XAMPP)

/var/www/html/ceaa_recursos/ (Apache en Linux)

2. Instalar dependencias Composer:

```
cd ceaa_recursos
```

```
composer install
```

3. Crear la base de datos:

```
mysql -u root -p -e "CREATE DATABASE sistema_ceaa CHARACTER SET  
utf8mb4 COLLATE utf8mb4_unicode_ci;"
```

4. Importar el esquema:

```
mysql -u root sistema_ceaa < sistema_ceaa.sql
```

5. Aplicar scripts de actualización (en orden cronológico):

```
mysql -u root sistema_ceaa < sql_updates/agregar_cambios_inventario.sql
```

```
mysql -u root sistema_ceaa < sql_updates/2026-03-  
02_operativa_bitacora_indices.sql
```

6. Configurar credenciales de BD en config/database.php.



7. Verificar permisos de escritura en:

`public/assets/uploads/perfiles/`

`public/pdf/`

8. Acceder a: http://localhost/ceaa_recursos/public/

ACTUALIZACIÓN DEL SISTEMA

1. Hacer respaldo de la BD antes de cualquier actualización:

```
mysqldump -u root sistema_ceaa > backup_AAAA-MM-DD.sql
```

2. Reemplazar archivos del proyecto (excluir vendor/ y config/database.php).

3. Si se añadieron nuevas dependencias Composer:

```
composer install
```

4. Si hay nuevos scripts en sql_updates/, aplicarlos:

```
mysql -u root sistema_ceaa < sql_updates/nuevo_script.sql
```

5. Si se actualizaron CSS o JS, incrementar el parámetro ?v=N en

las referencias dentro de views/dashboard.php para invalidar caché.



RESPALDO Y RECUPERACIÓN

Respaldo completo (BD + archivos):

```
mysqldump -u root sistema_ceaa > backup_sistema_ceaa.sql
```

```
xcopy /E /I ceaa_recursos ceaa_recursos_backup (Windows)
```

```
tar czf ceaa_recursos_backup.tar.gz ceaa_recursos/ (Linux)
```

Restaurar la BD:

```
mysql -u root sistema_ceaa < backup_sistema_ceaa.sql
```

Incluir en la política de respaldo los directorios:

public/pdf/ PDFs generados

public/assets/uploads/ Fotos de perfil